

UNIDAD 2 · SEMANA 2

Llamadas al sistema y E/S de bajo nivel

📅 Semana 2 · fechas por confirmar </> Laboratorio TD1 (resuelto + desafío IA)

~ 4 animaciones

MANUAL M2 · TEORÍA DE LA UNIDAD

Llamadas al sistema y E/S de bajo nivel

Lectura base de la unidad 2 (≈ 1.5 h). Cada sección termina con un enlace **Practica esto** → al ejercicio o a la animación que la ilustra. Las cifras salen de ejecuciones reales de los programas del TD1 en el equipo del curso (Linux 7.0, x86-64, glibc 2.43).

CONTENIDO

- | | | | |
|----|---|----|---|
| §1 | Modo usuario y modo núcleo | §4 | E/S de bajo nivel con <code>read</code> y <code>write</code> |
| §2 | Anatomía de una llamada al sistema | §5 | Cuánto cuesta una llamada |
| §3 | Observar: <code>man</code> , <code>strace</code> , <code>objdump</code> | §6 | Más allá: <code>vDSO</code> , <code>io_uring</code> , otras arquitecturas |

§1 Modo usuario y modo núcleo

El procesador tiene al menos dos **niveles de privilegio**. En x86-64 se llaman anillos: el núcleo corre en el anillo 0 (CPL 0) y los programas en el anillo 3 (CPL 3). En modo usuario, las instrucciones que tocan el hardware (entrada/salida, la tabla de páginas, desactivar interrupciones) están prohibidas: si un programa las intenta, el procesador genera una excepción y el núcleo lo termina.

Esa frontera es la base de todo lo demás: gracias a ella un programa no puede leer la memoria de otro, escribir en el disco saltándose los permisos ni apropiarse de la CPU. Cuando un programa necesita algo que solo el núcleo puede hacer, **lo pide**: eso es una llamada al sistema.

§2 Anatomía de una llamada al sistema

Cuando tu programa escribe `write(1, "hola\n", 5)`, en realidad llama a una función de glibc (la *envoltura*) que sigue la convención de Linux x86-64:

1. Pone el **número de la llamada** en el registro `rax` (1 para `write`, 0 para `read`, 110 para `getppid`) y los argumentos en `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`. Los datos no viajan en los registros: `rsi` lleva la *dirección* del texto.
2. Ejecuta la instrucción `syscall`: la CPU pasa a modo núcleo y salta a una dirección fija que el núcleo configuró al arrancar (`entry_SYSCALL_64`). El programa no puede elegir adónde saltar.

3. El núcleo usa `rax` como índice en `sys_call_table` y ejecuta el manejador (`ksys_write`), que valida el descriptor, copia los datos desde la memoria del programa (`copy_from_user`) y llama al controlador del dispositivo.
4. El resultado vuelve en `rax` y la instrucción `sysret` devuelve la CPU a modo usuario.
5. Si `rax` está entre `-4095` y `-1`, la envoltura lo interpreta como error: guarda el código en `errno` y devuelve `-1`. Por eso el patrón de todo el curso es `exit_if(rc == -1, "...")`, que muestra el error con `perror`.

Salida real

```
$ objdump -d --no-show-raw-insn /usr/lib/x86_64-linux-gnu/libc.so.6 | grep -A4 -E '^[0-9a-f]+ <getppid@>'
0000000000ff710 <getppid@GLIBC_2.2.5>:
    ff710:    endbr64
    ff714:    mov     $0x6e,%eax
    ff719:    syscall
    ff71b:    ret
```

Practica esto → animación A2.1 · Viaje de una llamada al sistema

§3 Observar: `man`, `strace`, `objdump`

Tres herramientas para investigar cualquier programa sin su código fuente:

- `man 2 write` documenta la llamada al sistema; `man 3 printf`, la función de biblioteca; `man 3p`, el estándar POSIX.
- `strace ./programa` muestra cada llamada al sistema con sus argumentos y su resultado; `-f` sigue a los hijos, `-r` agrega el tiempo entre llamadas y `-c` hace un resumen.
- `objdump -d` desensambla: así se ve la instrucción `syscall` dentro de `libc`.

Una sorpresa al usar `strace`: el programa `echo` del TD1 hace unas **30 llamadas antes de llegar a `main`**. Son del cargador dinámico, que abre `libc.so.6`, la mapea en memoria con `mmap` y protege sus páginas con `mprotect`.

Practica esto → animación A2.2 · `strace` en vivo

§4 E/S de bajo nivel con `read` y `write`

Todo proceso nace con tres **descriptores de archivo** abiertos: 0 (entrada estándar), 1 (salida estándar) y 2 (salida de error). El shell decide a qué apuntan: `< archivo`, `> archivo`, `2>&1` o una tubería. El programa no se entera: solo usa los números.

LLAMADA	DEVUELVE	CUIDADO CON
<code>read(fd, buf, n)</code>	bytes leídos (1...n), 0 al final del archivo, -1 en error	puede devolver <i>menos</i> de <code>n</code> (tubería, terminal, red, señal)
<code>write(fd, buf, n)</code>	bytes escritos (1...n), -1 en error	escritura parcial (tubería o socket llenos); <code>EINTR</code> si una señal la interrumpe

Por eso los programas del curso repiten: `read_exact` lee hasta completar lo pedido y `write_all` escribe hasta sacar todo. Además, `read` y `write` copian **bytes**, no texto: escribir un `long` copia sus 8 bytes en el orden de la máquina (*little-endian* en x86-64), y escribir una estructura copia también su relleno de alineación (24 bytes para `struct nopad`).

Salida real

```
$ ./write_number | od -A d -t x1
00000000 05 00 00 00 00 00 00 00 08 00 00 00 00 00 00 00
00000016
```

⚠ Error clásico: avanzar el puntero con lo que falta

El ejemplo de lectura de las diapositivas originales (B25, diapos. 51) hacía `length = length - byte; buf = buf + length;`. Con un archivo regular funciona —la primera lectura trae todo y `length` vale 0—, pero con una tubería o un socket deja huecos y escribe **fuera del búfer**. Lo correcto es `buf += byte`, reintentar ante `EINTR` y detenerse con 0. Míralo paso a paso en A2.4 y Pruébalo con `cat archivo | ./read_struct`.

Practica esto → TD1, ejercicios 3 a 7

§5 Cuánto cuesta una llamada

Cada ida y vuelta al núcleo tiene un costo fijo: cambiar de modo, guardar y restaurar registros, validar argumentos. El programa `bench_read` del TD1 lo mide copiando 8 MiB a `/dev/null` con búferes de 1 byte a 1 MiB. Estos son los resultados reales en el equipo del curso:

Salida real

```
$ ./bench_read      # 8 MiB file, buffers of 1 B to 1 MiB
buffer_bytes,read_calls,seconds,MB_per_s,ns_per_iteration
1,8388609,6.4984,1.3,775
4,2097153,1.5997,5.2,763
16,524289,0.3952,21.2,754
64,131073,0.0998,84.1,761
256,32769,0.0260,322.6,794
1024,8193,0.0075,1121.4,913
4096,2049,0.0025,3351.2,1222
16384,513,0.0010,8319.5,1966
65536,129,0.0008,10834.7,6002
262144,33,0.0007,11832.2,21484
1048576,9,0.0008,10287.9,90598
```

Con 1 byte por llamada se hacen 8,4 millones de iteraciones y la copia tarda segundos; con 64 KiB, 129 iteraciones y menos de un milisegundo. Ajustando el modelo $tiempo = llamadas \times c_0 + bytes \times c_1$ a las mediciones se obtiene un peaje de alrededor de **0,7 μ s por iteración** (`read` + `write`) y menos de 0,1 ns por byte copiado. Conclusión práctica: usa búferes de algunos KiB o más (la biblioteca de C lo hace por ti con `printf` / `fwrite`, unidad 4) y nunca leas de a un byte en un bucle.

Practica esto → animación A2.3 · El costo de una llamada

§6 Más allá: vDSO, `io_uring` y otras arquitecturas

- **vDSO.** Algunas "llamadas" muy frecuentes, como `clock_gettime`, no entran al núcleo: Linux mapea en cada proceso una pequeña biblioteca (vDSO) que lee la hora de una página compartida. Por eso no aparecen en `strace`.
- **`io_uring`** (Linux 5.1, 2019). Para programas que hacen muchísima E/S: el programa y el núcleo comparten dos colas en memoria; se pueden enviar cientos de operaciones con una sola llamada, o ninguna. La misma idea del búfer grande, llevada a las llamadas.
- **Otras arquitecturas.** En ARM de 64 bits la instrucción es `svc #0` y el número va en `x8`; en RISC-V es `ecall` con el número en `a7`. Los números cambian entre arquitecturas; la idea, no.
- **Observabilidad con eBPF.** Herramientas como `bpftrace` cuentan llamadas de todo el sistema con un costo mucho menor que `strace`, que detiene el proceso en cada llamada.